

GamutX

Strings

Python Fundamentals

Introduction to Strings

- What is a String?
- How to Create a String
- Importance of Strings in Programming

What is String?

- A string is just a sequence of characters. These characters can be:
 - Letters (A, b, Z, etc.)
 - Numbers (1, 42, 0)
 - Symbols (@, #, %)
 - Even spaces ()

There are three main ways to define a string using

Single Quotes

'Hello'

Double Quote

"Hello"

Triple Quotes

"""Hello""" or """"Hello""""

How to Create a String

- Think of a **string** like **text in a message** or **even like writing your name on a form**

```
name = "Alice"  
print(name) # Alice
```

String Literals

sequence of characters **enclosed in either single quotes (') or double quotes**

Variable Name

It is an **identifier** used to **label and store data in a program**

Different Ways to Write Strings

- Let's look at some other **ways to write a String**.

◆ Single Quotes

```
string1 = 'Hello, World!'
print(string1) # hello, World!
```

- useful **when your text doesn't have any apostrophes** or **single quotes** inside it.

◆ Triple Single Quotes

```
multi_line_string = '''This is a string
that spans multiple lines.'''

print(multi_line_string)

# This is a string
# that spans multiple lines.
```

◆ Double Quotes

```
string3 = "It's a sunny day"
print(string3) # It's a sunny day
```

- useful when your **text Does have any apostrophes** or **single quotes** inside it.

◆ Triple Double Quotes

```
another_multi_line_string = """Another way
to create multi-line strings."""

print(another_multi_line_string)

# Another way
# to create multi-line strings.
```

Both Single & Double Quotes are Useful for multi-line strings

Importance of Strings in Programming

◆ Data Representation

- Usage: Handle user input, file contents, and text in applications.
- Example: Storing user names like "Alice" or messages like "Welcome to the program!".

◆ User Interaction

- Usage: Collect user input or display messages and prompts.
- Example: Asking "What is your name?" or displaying "Thank you for your submission!".

◆ Text Processing

- Usage: Perform operations like searching, splitting, joining, and replacing text.
- Example: Searching for "error" in a log file, splitting a sentence into words, or replacing "foo" with "bar".

String Manipulation

- Concatenation (+)
- Repetition (*)
- Comma (,)

Concatenation (+)

- Concatenation means **joining two or more strings together**.
- We **use the + operator to do this**.

Basic Concatenation

```
first_name = "John"
last_name = "Doe"

full_name = first_name + last_name
print(full_name)
```

```
# Output: JohnDoe
```

Concatenation with Space

```
first = "hello"
second = "world"

result = first + " " + second
print(result)
```

```
# Output: 'hello world'
```

Concatenating Multiple Strings

```
first = "hello"
middle = "beautiful"
last = "world"

result = first + " " + middle + " " + last
print(result)
```

```
# Output: 'hello beautiful world'
```

- We **added a space (" ")** between the names.
- **+ does not add space automatically**

Repetition (*)

- Repetition creates a new string by repeating an existing string a specified **number of times**.
- Use the ***** operator to repeat a string.

Basic Repetition

```
word = "hello"  
print(word * 3) -----  
# Output : hellohellohello
```

Repeating Characters

```
stars = "*" * 10 -----  
print(stars) # Output: **********
```

Creating a Simple Divider

```
divider = "-" * 20 -----  
print(divider)  
# output : -----
```

Note

- **does not automatically insert space** between the arguments.
- If we **want spaces**, we include them in the string

Concept Practice Task

Handling Empty String

```
empty = "" * 5  
print(empty)
```

Zero Repetition

```
text = "Python" * 0  
print(text)
```

Handling Negative Values

```
negative_repeat = "Hello" * -3  
print(negative_repeat)
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS  
● PS C:\Users\HP\Desktop\tt> & "C:/Program Files/python.exe" c:/Users/HP/Desktop/tt/test.py  
○ PS C:\Users\HP\Desktop\tt>
```

Multiplying with Zero, Negative number, or with and empty string always return and empty output

Concept Practice Task

GamutX

Basic Repetition

```
# Repeat a string multiple times
result = "Hello " * 3
print(result) # Output: Hello Hello Hello
```

Repeating Characters in a String

```
char = "*"
pattern = (char + " ") * 5
print(pattern)
# output : * * * * *
```

Multiplying Strings with Variables

```
word = "Go"
times = 4
cheer = (word + " ") * times
print(cheer)
# output : Go Go Go Go
```

Comma (,)

- commas are **used to separate multiple arguments** within functions like print().
- It **automatically insert space between arguments**.

Using for Concatenations

```
word1 = "Hello"  
word2 = "World"  
  
print(word1, word2)
```

```
# Output: hello World
```

Listing Items Using Commas

```
print("Apples,", "Bananas,", "Cherries")
```

```
#Output: Apples, Bananas, Cherries
```

- This is useful for **printing multiple values without using + or adding spaces manually**.
- Also **Displaying multiple pieces of data clearly**.

String Operations

- Comparison Operators
- ASCII TABLE
- Membership Operators

Comparison Operators

- Comparison **operators** are used to compare two strings.
- They determine the **relationship between two strings based on their lexicographic order**.
- The **result** is always either **True** or **False** (Boolean values).
- Python is **Case-sensitive**: "A" is not the same as "a".
- In String **Comparisons** are done character by character (**left to right**).

List of Comparison Operator

== Equal to

!= Not equal to

> Greater than

< Less than

>= Greater than or equal to

<= Less than or equal to

ASCII TABLE

GamutX

- Characters are **compared by their ASCII values.** → **American Standard Code for Information Interchange**

a is **not equal** to A

Lowercase	ASCII Code	Uppercase	ASCII Code
a	97	A	65
b	98	B	66
c	99	C	67
d	100	D	68
e	101	E	69
f	102	F	70
g	103	G	71
h	104	H	72
i	105	I	73
j	106	J	74
k	107	K	75
l	108	L	76
m	109	M	77

Lowercase	ASCII Code	Uppercase	ASCII Code
n	110	N	78
o	111	O	79
p	112	P	80
q	113	Q	81
r	114	R	82
s	115	S	83
t	116	T	84
u	117	U	85
v	118	V	86
w	119	W	87
x	120	X	88
y	121	Y	89
z	122	Z	90

How Python Compares Strings

◆ First Characters Are Different

```
print("banana" < "cherry")  
# Output: True
```

What's Happening:

- Python looks at the first character of each string:
 - 'b' (ASCII 98)
 - 'c' (ASCII 99)

Rule

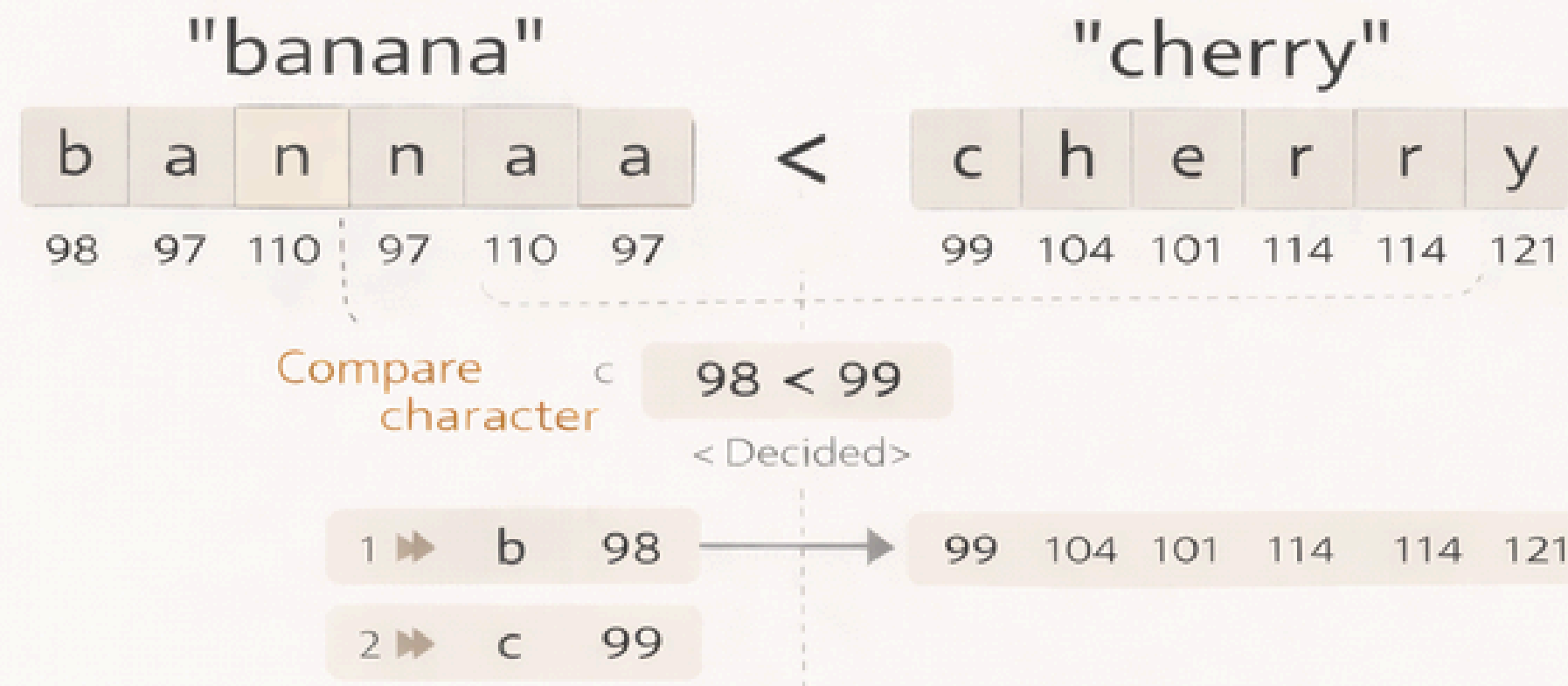
- Python compares **from left to right**.
- **As soon as it finds a difference**, it compares those **characters by their ASCII (Unicode) value**.
- It **does not** check the rest of the string.

Conclusion:

- 'b' < 'c' → So "banana" is less than "cherry".

Lexicographical Comparison

Strings are compared character by character using ASCII values,



✓ First different character decides

◆ Same Start, Difference Comes Later

```
print("banana" < "bherry")  
# Output: True
```

What's Happening:

- First character:
 - 'b' vs 'b' → same
- Second character:
 - 'a' vs 'h' → different
- Python compares their ASCII values:
 - 'a' = 97
 - 'h' = 104
- Since **97 < 104**, Python considers "banana" to be less than "bherry".

◆ Strings Start the Same, But Differ Later

```
print("file1" < "file2")  
# Output: True
```

What's Happening:

- Python compares character by character:
 - 'f' == 'f'
 - 'i' == 'i'
 - 'l' == 'l'
 - 'e' == 'e'
 - '1' vs '2'

ASCII Values:

- '1' = 49
- '2' = 50

Conclusion:

- '1' < '2' → So "file1" is less than "file2".

◆ One String is a Shorter Version of the Other

```
print("apple" < "apples")  
# Output: True
```

What's Happening:

- Python compares each character:
 - 'f' == 'f'
 - 'i' == 'i'
 - 'l' == 'l'
 - 'e' == 'e'
 - '1' vs '2'
- At this point, **"apple"** ends, but **"apples"** still has **'s'**.

Rule → If all compared characters are equal, the **shorter string** is considered smaller.

Why: → Python treats the shorter string as **"less than"** because it runs out of characters first.

Less Than <

```
result = "apple" < "banana"
print(result) # True
```

Aspect	Description
Operation	< (less than)
Strings	"apple" vs "banana"
ASCII	"apple": 97 112 112 108 101
Values	"banana": 98 97 110 97 110 97
Result	True

Greater Than >

```
result = "cherry" > "banana"
print(result) # True
```

Aspect	Description
Operation	> (greater than)
Strings	"cherry" vs "banana"
ASCII	"cherry": 99 104 101 114 114 121
Values	"banana": 98 97 110 97 110 97
Result	True

Equal To ==

```
result = "grape" == "grape"
print(result) # True
```

Aspect	Description
Operation	== (equal to)
Strings	"grape" vs "grape"
ASCII	"grape": 103 114 97 112 101
Values	"grape": 103 114 97 112 101
Result	True

Not Equal To !=

```
result = "kiwi" != "apple"
print(result) # True
```

Aspect	Description
Operation	!= (not equal to)
Strings	"kiwi" vs "apple"
ASCII	"kiwi": 107 105 119 105
Values	"apple": 97 112 112 108 101
Result	True

Greater Than or Equal To >=

```
result = "mango" >= "apple"
print(result) # True
```

Aspect	Description
Operation	>= (greater than or equal to)
Strings	"mango" vs "apple"
ASCII	"mango": 109 97 110 103 111
Values	"apple": 97 112 112 108 101
Result	True

Less Than or Equal To <=

```
result = "apple" <= "orange"
print(result) # True
```

Aspect	Description
Operation	<= (less than or equal to)
Strings	"apple" vs "orange"
ASCII	"apple": 97 112 112 108 101
Values	"orange": 111 114 97 110 103 101
Result	True

Summary of Types of Errors

Step

- **Start from the left**
- **Continue if equal**
- **Stop when different**
- **If all equal so far**

What Python Does

- **Compare one character at a time**
- **Keep checking until characters differ**
- **Compare based on ASCII/Unicode value**
- **string is considered smaller**

How String Comparison Works

1- Character-by-Character Check

- Python compares **strings one character at a time, from left to right.**
- If **characters are different**, the **one with the smaller letter** (alphabetically) **is smaller.**
 - **Example :** "apple" < "banana" → because 'a' < 'b'

2- Handling Different Lengths

- If the **beginning of both strings is the same**, the **shorter one is smaller.**
- "cat" < "catalog" → because "cat" **is shorter** but matches the beginning of "catalog"

3- ASCII Values

- Python **uses ASCII codes to compare letters.** Each letter has a number:
- **'A' = 65, 'a' = 97** (Capital letters come before lowercase!)

What is Lexicographic Order?

- It is **a way of sorting strings based on the alphabetical order of their characters,**
- similar to **how words are ordered in a dictionary.**
- Python uses this to compare strings.

Membership Operators

- Membership operators are **used to check whether a specific substring exists inside another string.**
- They check for the **exact sequence of characters, not just individual letters.**

There are two types of Operator

Operator	Meaning	Returns
in	True if the value exists in the string	True / False
not in	True if the value does not exist in the string	True / False

Using in

Substring Found

```
# Check if a substring exists within string.
text = "hello world"
result = "world" in text

print(result)
# Output: True
```

Case Sensitivity

```
# The 'in' operator is case-sensitive.
text = "Hello World"
result = "world" in text

print(result)
# Output: False
```

Using not in

Substring Not Found

```
# Check if a substring does NOT exist .
text = "hello world"
result = "Python" not in text

print(result)
# Output: True
```

Special Characters

```
# Check if a special character does NOT exist.
text = "hello@world"
result = "#" not in text

print(result)
# Output: True
```

Concept Practice Task

GamutX

Substring Found

```
# Check if a substring exists within the string.
text = "hello world"
result = "world" in text

print(result)
# Output: True
```

Special Characters

```
# Check if a special character does NOT exist within the string.
text = "hello@world"
result = "#" not in text

print(result)
# Output: True
```

Substring Not Found

```
# Check if a substring does NOT exist within the string.
text = "hello world"
result = "Python" not in text

print(result)
# Output: True
```

Case Sensitivity

```
# The 'in' operator is case-sensitive.
text = "Hello World"
result = "world" in text

print(result)
# Output: False
```

String Escape Characters

- What is Escape Characters?
- Tab - `\t`
- Newline (`\n`)
- Single Quote (`\'`)
- Backslash (`\\`)

What is Escape Characters

- Membership operators are **used to check whether a specific substring exists inside another string.**
- They check for the **exact sequence of characters, not just individual letters.**

✗ Problem

```
print("I am Raj and I am from "India". ")
```

```
⊗ PS C:\Users\HP\Desktop\tt> & "C:/Program Files/python.exe" c:  
File "c:\Users\HP\Desktop\tt\test.py", line 1  
    print("I am Raj and I am from "India". ")  
                                   ^  
SyntaxError: invalid syntax  
○ PS C:\Users\HP\Desktop\tt>
```

Why This Happens

- Python **starts reading the sentence after the first "**
- When it **sees the next "** before "India", it **thinks the sentence is over**
- So the **rest (like "India".) confuses Python**

How to Fix It ?

```
print("I am Raj and I am from \"India\". ")  
#Output: I am Raj and I am from "India".
```

We can Fix this using backslash (\) to escape the quote: \"

Other Fix Options

Use single quotes outside

```
print('I am Raj and I am from "India".')
```

Use triple quotes for longer text

```
print("""I am Raj and I am from "India".""")
```

Tab - \t

- The `\t` escape character adds a tab space in the string — like pressing the Tab key on a keyboard.

Basic Tab Usage

```
# Using \t to insert a tab space between words.
text = "Hello\tWorld"

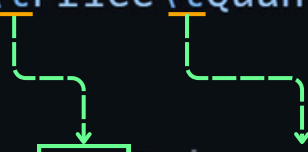
print(text)
# Output: Hello    World
```



Multiple Tabs

```
# Using multiple \t characters to add tab spaces.
text = "Item\tPrice\tQuantity"

print(text)
# Output: Item    Price    Quantity
```



Tab in String Formatting

```
# Using \t to format and align data in output.
text = "Product\tPrice\nApple\t$1.00\nBanana\t$0.50"

print(text)
```

Output:

Product	Price
Apple	\$1.00
Banana	\$0.50

`\n` - Newline

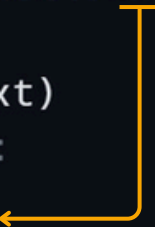
`\t` - Tab

Newline - \n

- The \n escape character creates a new line in the output.

Basic Newline Usage

```
# Using \n to create a new line between words.  
text = "Hello\nWorld"  
  
print(text)  
# Output:  
# Hello  
# World
```



Multiple Newlines

```
# Using multiple \n characters to create several blank lines.  
text = "Hello\n\nWorld"  
  
print(text)  
# Output:  
# Hello  
#  
# World
```



Newline in Multiline Strings

```
# Using \n within a multiline string.  
text = """This is line 1\nThis is line 2\nThis is line 3"""  
  
print(text)  
# Output:  
# This is line 1  
# This is line 2  
# This is line 3
```



Newline in Print Statements

```
# Combining separate print statements with \n.  
print("Line 1\nLine 2\nLine 3")  
# Output:  
# Line 1  
# Line 2  
# Line 3
```

Single Quote (\')

Basic Single Quote Escape

```
# Escaping a single quote inside a string .
text = 'It\'s a sunny day.'

print(text)
# Output: It's a sunny day.
```

Single Quote in Double-Quoted String

```
# Using a string enclosed by double quotes.
text = "It's a sunny day."

print(text)
# Output: It's a sunny day.
```

Escaping Single Quote in Multiline String

```
# Using \' in a multiline string to include a single quote.
text = """This is a string that includes a single quote:
        It\'s part of the text."""

print(text)
# Output: This is a string that includes a single quote: It's part of the text.
```

Single Quote in JSON-Like Data

```
# Using \' to represent a single quote in JSON-like data.
json_data = '{"message": "It\'s a great day!"}'

print(json_data)
# Output: {"message": "It's a great day!"}
```


Backslash- \\

- To print a backslash itself, we need to escape it using \\.

Basic Backslash Escape

```
# Escaping a backslash in a string.
text = "This is a backslash: \\"
print(text)
# Output: This is a backslash: \
```



Backslash in File Paths

```
# Using backslashes in a Windows file path.
file_path = "C:\\Users\\John\\Documents\\file.txt"
print(file_path)
# Output: C:\Users\John\Documents\file.txt
```



Backslash for Line Continuation

```
# Using a backslash to continue a string on the next line.
text = "This is a long string that \
spans multiple lines."
print(text)
# Output: This is a long string that spans multiple lines.
```



This will convert it in single line

Special Characters with Backslash

```
# Using backslash to escape special characters.
text = "Here is a quote: \"Hello!\" and a backslash: \\"
print(text)
# Output: Here is a quote: "Hello!" and a backslash: \
```



String Formatting

- % Operator
- {} str.format() Method
- {0} str.format() Positional Arguments
- {name} str.format() Keyword Arguments

String Formatting Demo

GamutX

```
name = "Alice"
char = "A"
second_name = "Eve"
```

1. Using the % Operator (Old Style)

```
print("My name is %s, and my char is %s" % (name, char))
```

Output: My name is Alice, and my char is A

2. Using .format() with basic placeholders

```
print("My name is {}, and my char is {}".format(name, char))
```

Output: My name is Alice, and my char is A

3. Using .format() with positional arguments

```
print("My name is {1}, and my char is {0}".format(name, char))
```

Output: My name is A, and my char is Alice

4. Using .format() with keyword arguments

```
print("My name is {first}, and my char is {second}. My second name is {third}".format(
    first=name,
    second=char,
    third=second_name
))
```

Output: My name is Alice, and my char is A. My second name is Eve

% Operator

- The % operator allows us to place values into a string using placeholders.
- Syntax
 - `formatted_string = "format string" % (values)`

List of Common Placeholders

%s → String formatter

%f → Floating-point number

%d → Integer formatter

%c → Character formatter

%s String

```
name = "Alice"
formatted_string = "Hello, %s!" % name
print(formatted_string)
# Output: Hello, Alice!
```

The % symbol is used to insert values into a string

%d Integer

```
age = 30
formatted_string = "I am %d years old." % age
print(formatted_string)
# Output: I am 30 years old.
```

%f Floating-point number

```
temperature = 23.4567
formatted_string = "The temperature is %.2f°C." % temperature
print(formatted_string)
# Output: The temperature is 23.46°C.
```

- %f is used for float values.
- %.2f means round to 2 decimal places.

Multiple Values in One String

```
name = "Bob"
age = 25
height = 175.5
formatted_string = "Name: %s, Age: %d, Height: %.1f cm" % (name, age, height)
print(formatted_string)

# Output: Name: Bob, Age: 25, Height: 175.5 cm
```

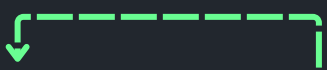
A diagram with three dashed arrows. A green arrow starts from the variable 'name' and points to the '%s' placeholder. An orange arrow starts from the variable 'age' and points to the '%d' placeholder. A red arrow starts from the variable 'height' and points to the '%.1f' placeholder.

- We can use different placeholders together and pass multiple values as a tuple.

%c Character formatter

```
char_code = 65
formatted_string = "The character is %c." % char_code
print(formatted_string)

# Output: The character is A.
```

A green dashed arrow starts from the variable 'char_code' and points to the '%c' placeholder.

- %c converts a number to its corresponding ASCII character.

{ str.format() Method

- The **format() method** is used to **format strings by replacing placeholders {} with corresponding values**.
- It allows you to **substitute variables or values into placeholder**.

```
"{} {} {}".format(value1, value2, value3)
```

You can place values inside the {} by passing them to the format() method.

These are placeholders for values you want to insert into the string.

Basic Placeholder Replacement

```
name = "Python"
age = 10
message = "My name is {} and I'm {} years old.".format(name, age)
print(message)

# Output: My name is Python and I'm 10 years old.
```

- The **first {}** gets replaced with the **value** of name, which is **"Python"**.
- The **second {}** gets replaced with the value of age, which is **10**.

{0} str.format() Positional Arguments

- In Python, the `format()` method lets you choose which value goes into which placeholder using **numbers inside {}**.
- This helps when **you want values to appear in a specific order**.
- Each `{}` can have a **number** that tells **Python which value to use from the format()**.

The diagram illustrates the mapping of positional arguments to placeholders in a Python format string. The code `print('{0} {1} cost ${2}'.format(6, 'bananas', 1.74))` is shown. A purple dashed bracket under the placeholders `{0}`, `{1}`, and `{2}` is labeled "Template". A white arrow points to the `.format` method, labeled "Method". A blue dashed bracket under the arguments `6`, `'bananas'`, and `1.74` is labeled "Positional Arguments". Colored dashed arrows show the mapping: a green arrow from `{0}` to `6`, an orange arrow from `{1}` to `'bananas'`, and a red arrow from `{2}` to `1.74`. Additionally, a green dashed line connects the `{0}` placeholder to the `1.74` argument, and an orange dashed line connects the `{1}` placeholder to the `6` argument, likely representing a swap or a specific ordering rule.

```
print('{0} {1} cost ${2}'.format(6, 'bananas', 1.74))
```

Template Method Positional Arguments

```
# Output: 6 bananas cost $1.74
```


Concept Practice Task

GamutX

Basic Positional Arguments

```
# Using positional arguments to format a string.
text = "{0} is {1} years old.".format("Alice", 30)

print(text)
# Output: Alice is 30 years old.
```

↓ 0 ↓ 1

Reusing Positional Arguments

```
# Reusing positional arguments multiple times.
text = "{0} loves {1}. {0} enjoys {1} a lot."
      .format("Bob", "reading")

print(text)
# Output: Bob loves reading. Bob enjoys reading a lot.
```

Index Reordering

```
# Reordering positional arguments in the format string.
text = "{1} is the name of the person. {0} is their age."
      .format(25, "David")

print(text)
# Output: David is the name of the person. 25 is their age.
```

↓ 0 ↓ 1

Nested Formatting

```
# Combining positional arguments with nested formatting.
text = "The price of {0} is {1:.2f}. With tax, it becomes {2:.2f}."
      .format("a book", 12.49, 12.49 * 1.07)

print(text)
# Output: The price of a book is 12.49. With tax, it becomes 13.36.
```

↓ 0 ↓ 1 ↓ 2

{name} str.format() Keyword Arguments

- In `str.format()`, keyword arguments allow us to use named placeholders in the string instead of relying on positions.
- This improves clarity, especially when:
 - There are many values
 - The order of values isn't obvious
 - You want the code to be easier to read



```
print('{quantity} {item} cost ${price}'.format(  
    quantity=6,  
    item='bananas',  
    price=1.74  
))
```

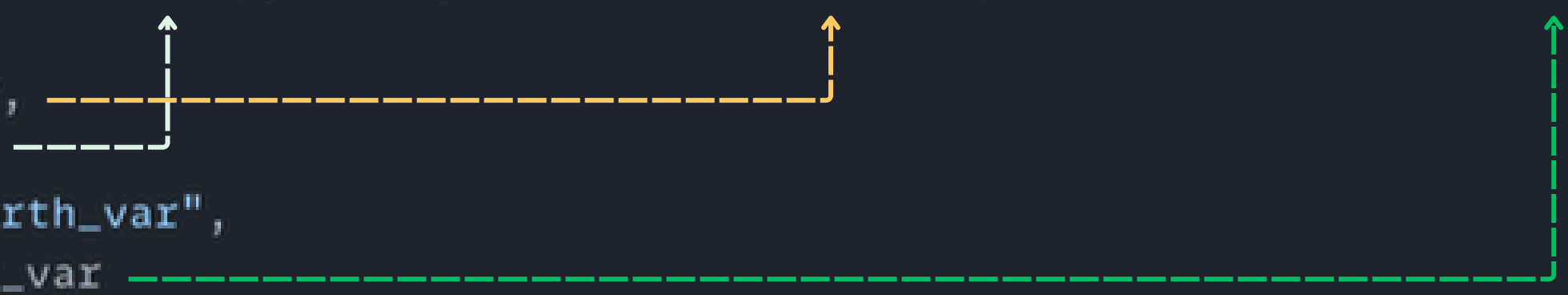
Keyword Arguments

```
# Output: 6 bananas cost $1.74
```

Advanced Example

```
name = "Alice"
char = "A"
first_var = "Eve"

# Using str.format() with keyword arguments
print("My name is {third_var}, and my char is {second_var}.\nSecond name: {fifth_var}".format(
    first_var=name,
    second_var=char,
    third_var="12",
    fourth_var="fourth_var",
    fifth_var=first_var
))
```



```
# Output
# My name is 12, and my char is A.
# Second name: Eve
```


Concept Practice Task

GamutX

Basic Keyword Arguments

```
# Using keyword arguments to format a string.
text = "{name} is {age} years old.".format(name="Alice", age=30)
print(text)
# Output: Alice is 30 years old.
```

Reusing Keyword Arguments

```
# Reusing the same keyword argument multiple times in the string.
text = "{name} loves {hobby}. {name} also enjoys {hobby} on weekends."
      .format(name="Bob", hobby="cycling")

print(text)
# Output: Bob loves cycling. Bob also enjoys cycling on weekends.
```

Index Reordering

```
# Reordering keyword arguments in the format string.
text = "{city} is a beautiful city. {name} is exploring {city}."
      .format(name="David", city="Barcelona")

print(text)
# Output: Barcelona is a beautiful city. David is exploring Barcelona.
```

Nested Formatting

```
# Combining keyword arguments with nested formatting.
text = "The price of {item} is ${price:.2f}. With tax, it becomes ${total:.2f}."
      .format(item="a book", price=12.49, total=12.49 * 1.07)

print(text)
# Output: The price of a book is $12.49. With tax, it becomes $13.36.
```

Key Points

Positional Arguments

- {0}, {1} are number placeholders.
- Numbers show the order of values.
- Values go inside format() in the same order.
- Counting starts from 0.

Example:

- "I have {0} {1}".format(3, "apples") → I have 3 apples

Keyword Arguments

- {name}, {item} are named placeholders.
- Names match the keys given in format().
- Order doesn't matter — only the name matters.
- Easier to read and understand.

Example:

- "Hello, {name}".format(name="John") → Hello, John

Note:

- [Combining Positional and Keyword Arguments] We cover this topic when we done with Functions

String Interpolation

- What is Interpolation ?
- Types of String Interpolation
- F-Strings (Formatted Strings)
- R-Strings (Raw Strings)

What is Interpolation ?

- Interpolation means **inserting values into a string**.
- It lets you create sentences or messages that change based on variables.
- In Python, the most common way to do this is using f-strings.

Think of it like filling in the blanks:

- **Example:**
 - name = "Alice"
 - **"Hello, [name]"** → become → **"Hello, Alice"**

Why It Matters

- In VFX work, we often need to create file names, paths, or log messages based on variables.
- String interpolation helps us build those strings clearly and efficiently.

Types of String Interpolation

Basic String Interpolation

- This is when we insert **variable values** into a string.

```
name = "Alice"  
age = 30  
info = f"My name is {name} and I am {age} years old."  
print(info)
```

Output:

My name is Alice and I am 30 years old.

- ✓ Both **{name}** and **{age}** are replaced by their values.

Interpolation with Expressions

- We can also place **calculations or expressions** directly inside the curly braces.

```
price = 49.99  
quantity = 3  
total_cost = f"The total cost is {price * quantity} dollars."  
print(total_cost)
```

Output:

The total cost is 149.97 dollars.

- ✓ **price * quantity** is calculated first, then inserted into the string.

F-Strings (Formatted Strings)

F-strings allow us to **insert variables and expressions directly inside a string, using {}**.

Syntax

```
f"some text {expression} more text"
```

Prefix the string with f.

Use curly braces {} to include variables or expressions directly within the string.

Key Points

- **Prefix with f:** Start your string with f to use f-strings.
- **Embed Expressions:** Include variables and expressions directly in {}.
- **Formatting:** Use format specifiers inside {} for number formatting and more.
- **Readability:** F-strings are often easier to read and write compared to older formatting methods.

Concept Practice Task

Using Expressions Inside F-Strings

```
a = 5
b = 10
print(f"The sum of {a} and {b} is {a + b}")
# output : The sum of 5 and 10 is 15
```

Formatting Numbers in F-Strings

```
pi = 3.14159265359
print(f"Pi to 3 decimal places: {pi:.3f}")
# Output : Pi to 3 decimal places: 3.142
```

R-Strings (Raw Strings)

- **Raw strings**, denoted by **prefixing the string with r or R**
- used to **handle special characters** in strings **without needing to escape them**.

Syntax:

```
r"some text with \backslashes"
```

How it Work?

- Add **r** before the string.
- **Backslashes** are treated as **literal characters**, not escape sequences.

Example

```
raw_string = r"This is a raw string: C:\Users\Name\Documents"  
print(raw_string)
```

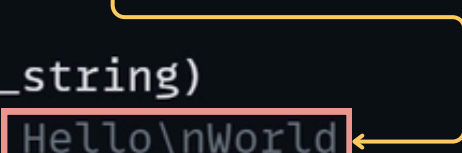
Explanation:

- The **r** in front of the string makes it a raw string.
- In raw strings, backslashes (****) are taken literally.
- This is useful when your string has backslashes, like in file paths.
- Normally, backslashes are used for escape characters (like **\n** for newline), but raw strings ignore that.

Concept Practice Task

Basic Raw String

```
# backslashes are treated as literal characters.  
raw_string = r"Hello\nWorld"  
  
print(raw_string)  
# Output: Hello\nWorld
```

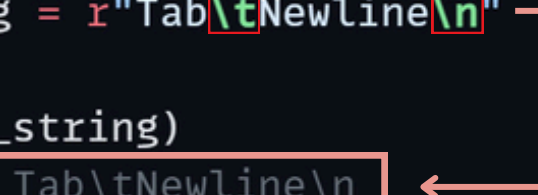


Raw String for File Paths

```
# Raw string used to represent a Windows file path.  
file_path = r"C:\Users\John\Documents\file.txt"  
  
print(file_path)  
# Output: C:\Users\John\Documents\file.txt
```

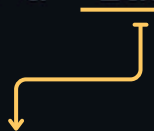
Raw String with Escape Sequences

```
# Escape sequences are printed literally in R strings.  
raw_string = r"Tab\tNewline\n"  
  
print(raw_string)  
# Output: Tab\tNewline\n
```



Raw String with Quotes Inside

```
# Raw strings with single or double quotes.  
raw_string = r'This is a "raw" string with quotes.'  
  
print(raw_string)  
# Output: This is a "raw" string with quotes.
```



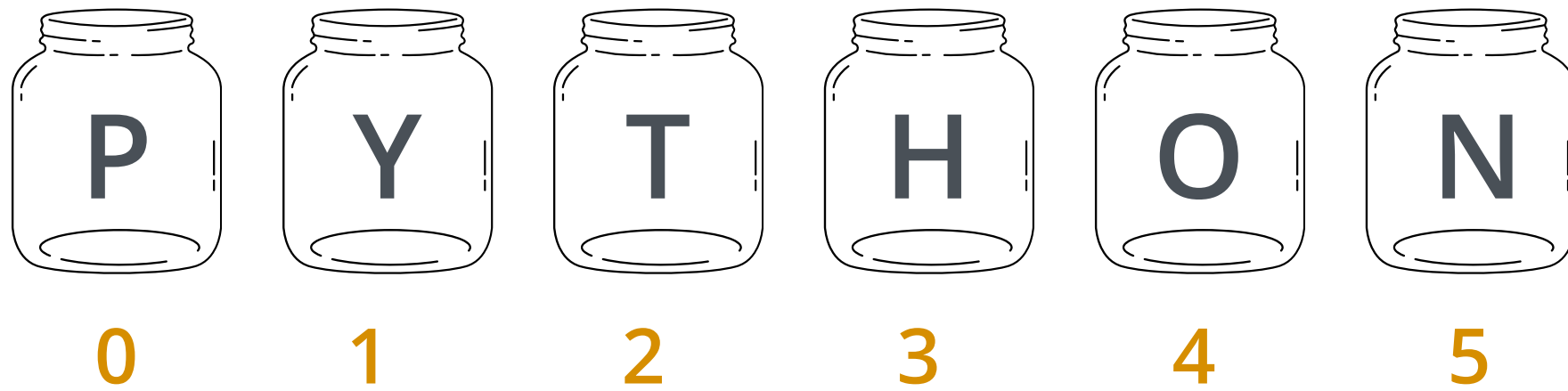
String Indexing

- What is indexing?
- Understanding - Indexing
- Types of Indexing
- Positive Indexing
- Negative Indexing

What is Indexing?

- In Python, **indexing means accessing individual characters in a string by their position.**
- Python starts **counting from 0**. So:
 - The **first character** has index **0**
 - The **second character** has index **1**, And so on...

word = "Python"



Start Here

Ends here

Understanding - Indexing

```
word = "Python"
first_letter = word[0]
print(first_letter) # Output: 'P'

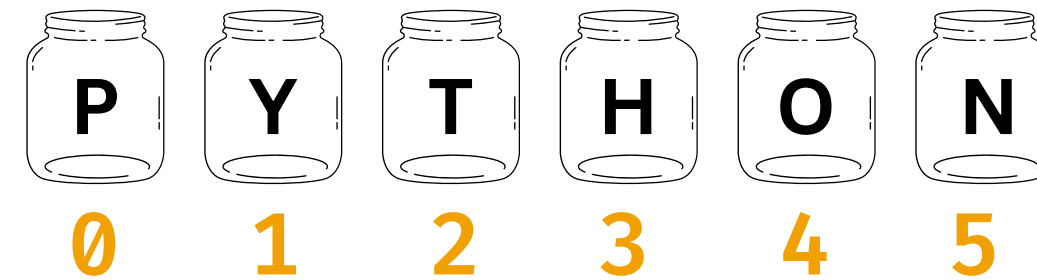
fourth_letter = word[3]
print(fourth_letter) # Output: 'h'
```

🔑 Important:

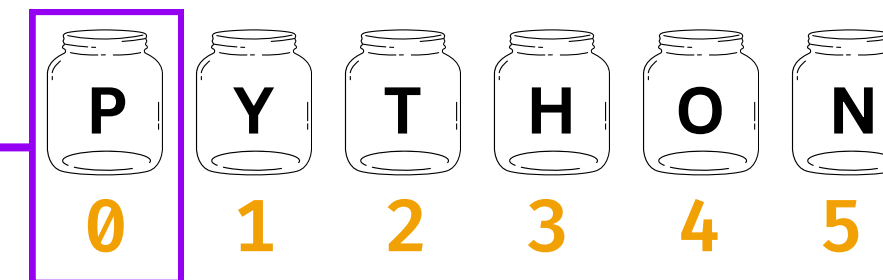
When using indexing in Python:

- We use square brackets []
- The index number goes inside the brackets

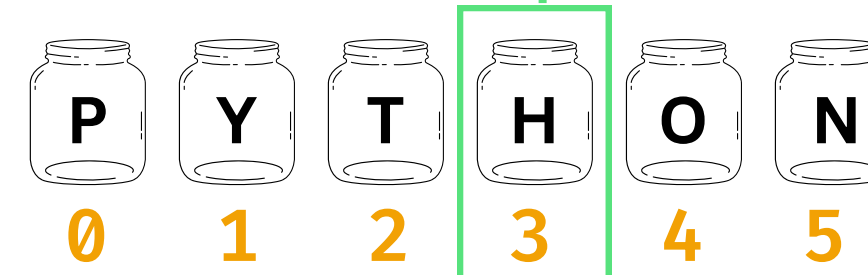
STEP 01



STEP 02



STEP 03

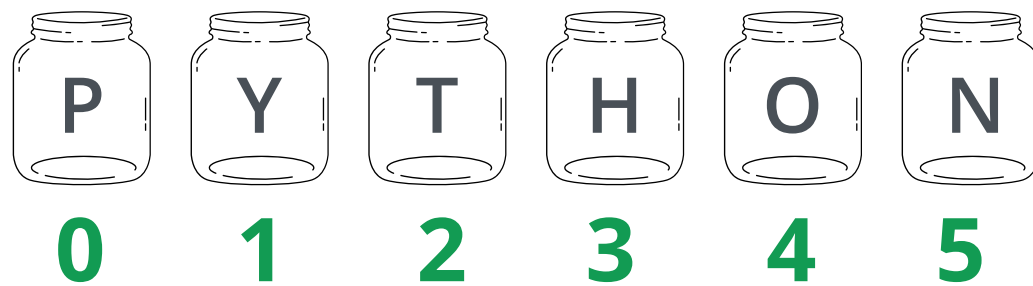


Types of Indexing

GamutX

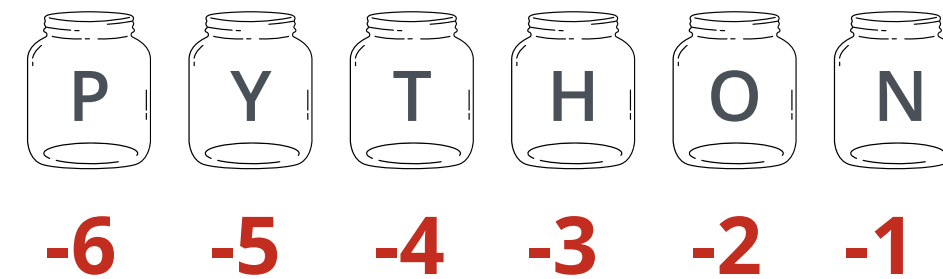
INDEXING

Positive Indexing



Positive Index : **Starts from 0 to n-1.**

Negative indexing



Negative Index : **Starts from -1 to -n.**

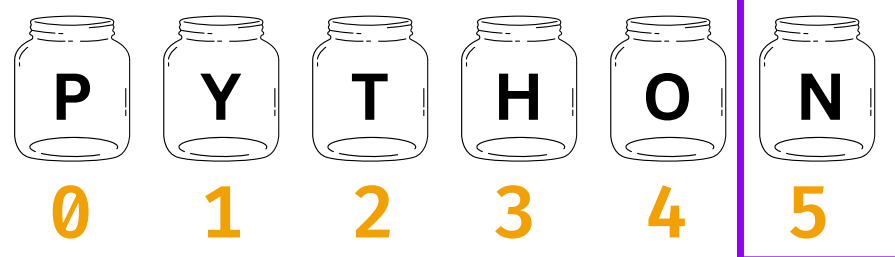
Positive Indexing

- Starts from the left side of the string
- Index values begin at 0 and go up to n-1, where n is the length of the string

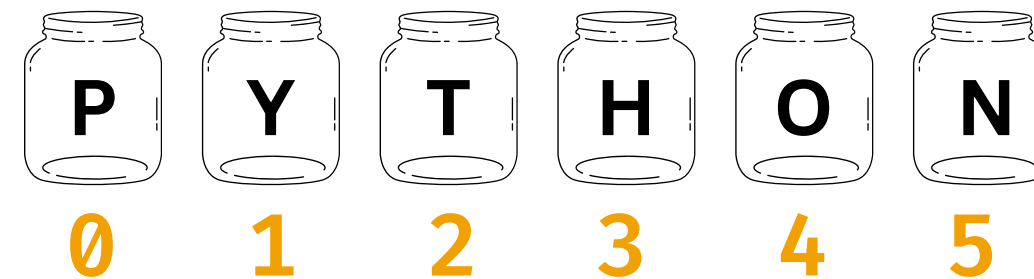
```
word = "Python"
```

```
print(word[0]) # Output: 'P'  
print(word[5]) # Output: 'M'
```

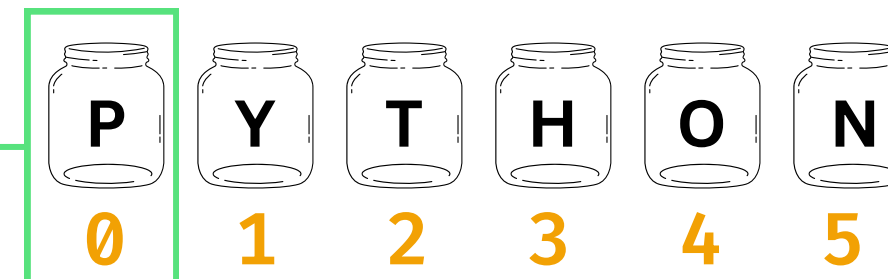
STEP 03 - Output



STEP 01



STEP 02



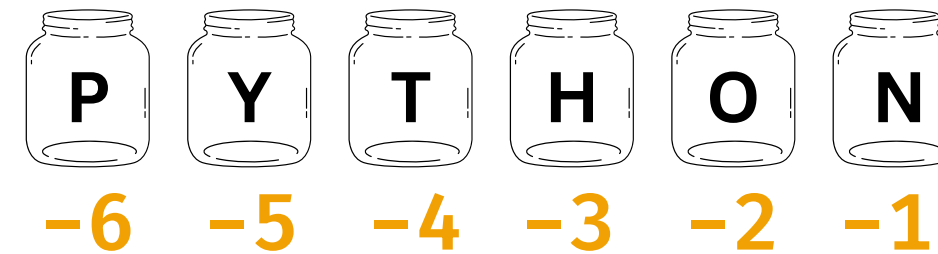
Negative Indexing

- Starts from the right side (end of the string)
- Index values begin at -1 and go backwards to -n
- helpful when working from the end of the string, especially during slicing

```
word = "Python"
```

```
print(word[-1]) # Output: 'n'  
print(word[-2]) # Output: 'o'
```

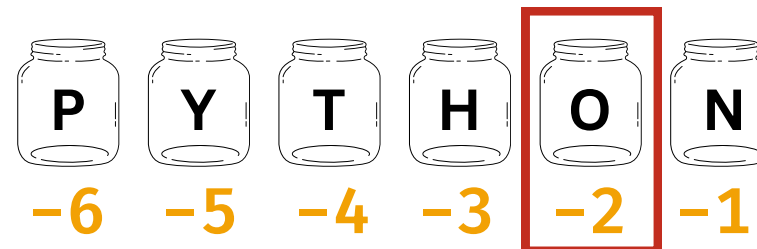
STEP 01



STEP 02



STEP 02



- -1 refers to the last character
- -2 to the second last, and so on.

String Methods

- What are String Methods?
- Commonly Used Methods

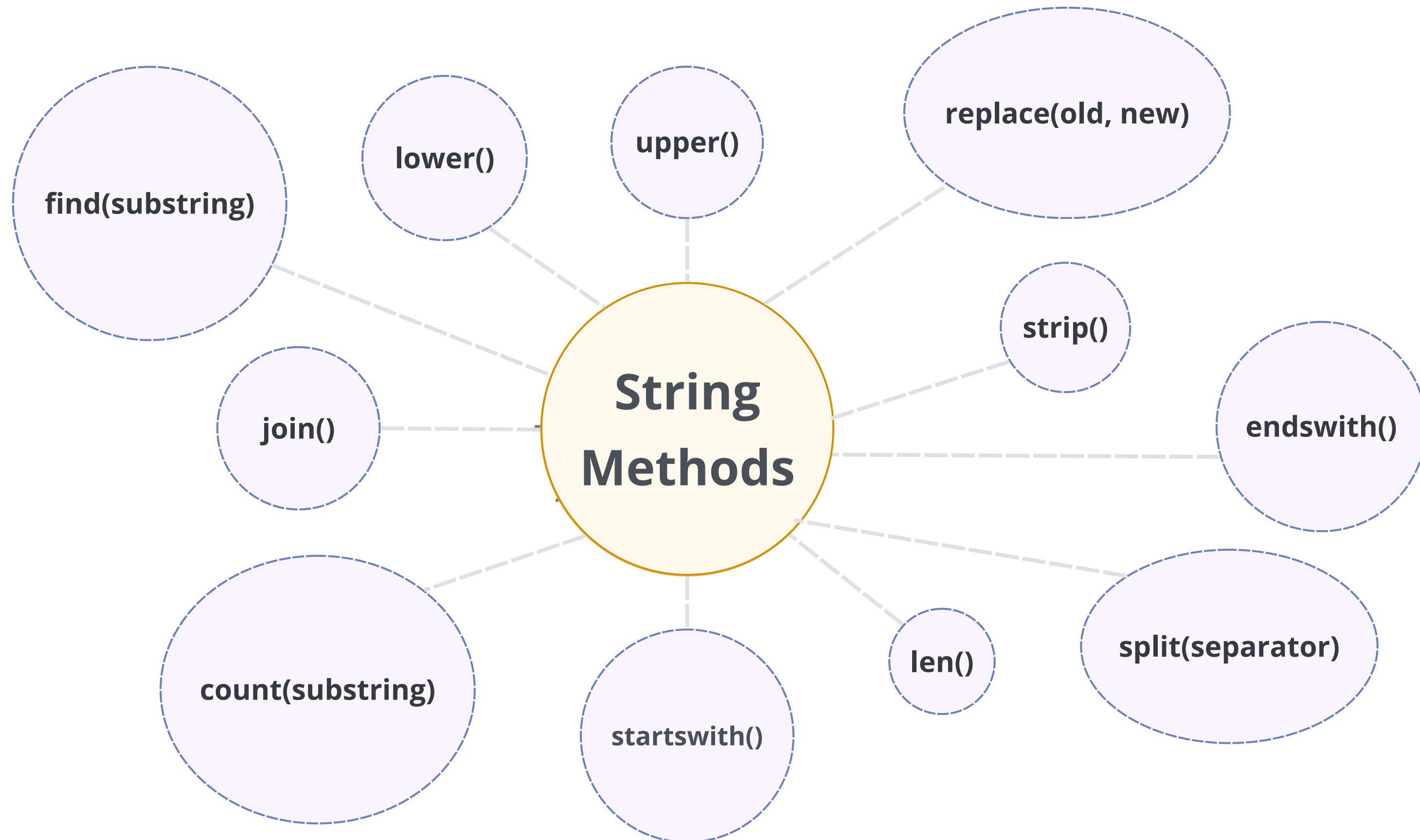
What are String Methods?

- String methods are **built-in functions in Python that are used to manipulate or query strings.**
- They make it easy to **perform common operations like:**
 - **Changing the case of letters**
 - **Finding parts of a string**
 - **Replacing text**
 - **Checking conditions (like if a string is numeric)**
- These methods help us work with text efficiently.



Commonly Used Methods

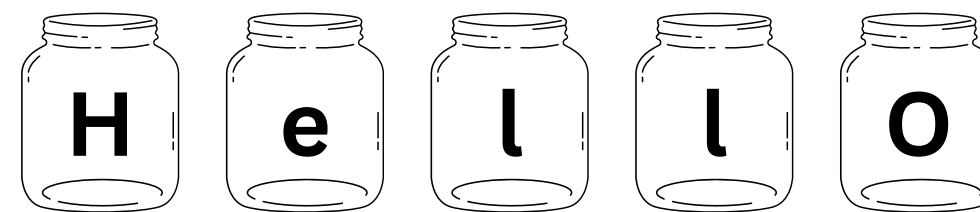
GamutX



len()

- It is **used to count total number of characters of a string**.
- **Note:** len() is not a string method, but a **built-in function in Python**.

```
my_string = "Hello"  
length = len(my_string)  
print(length) # 5
```



0

1

2

3

4

Python Indexing

1

2

3

4

5

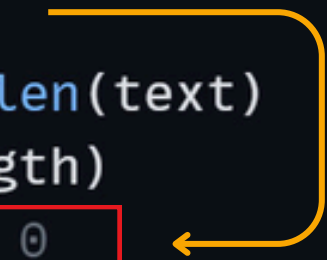
Human Counting

Concept Practice Task

GamutX

Empty String

```
1 # Get the length of an empty string.
2 text = ""
3 length = len(text)
4 print(length)
5 # Output: 0
```



String with Spaces

```
1 # Spaces with character in a String.
2 text = " __hello__ "
3 length = len(text)
4 print(length)
5 # Output: 11
```



String with Special Characters

```
1 # Special char in the string len.
2 text = "@hello@"
3 length = len(text)
4 print(length)
5 # Output: 7
```



Length of Numeric Characters

```
1 # Numbers in a string counted as Char.
2 text = "123456"
3 length = len(text)
4 print(length)
5 # Output: 6
```



human counting 1 2 3 4 5 6

Indexing in Python 0 1 2 3 4 5

upper()

- Converts all letters in a string to uppercase.

Basic Usage

```
# Convert a simple string to uppercase.  
text = "hello world"  
uppercase_text = text.upper()  
print(uppercase_text)  
# Output: 'HELLO WORLD'
```

Mixed Case Handling

```
# Convert a mixed case string to uppercase.  
text = "PyThoN Is FuN"  
uppercase_text = text.upper()  
print(uppercase_text)  
# Output: 'PYTHON IS FUN'
```

Special Characters Handling

```
# It does not affect numbers or special characters.  
text = "Welcome to 2024!"  
uppercase_text = text.upper()  
print(uppercase_text)  
# Output: 'WELCOME TO 2024!'
```

Empty String

```
# Applying upper() on an empty string.  
text = ""  
uppercase_text = text.upper()  
print(uppercase_text)  
# Output: ''
```


Concept Practice Task

String with Spaces

```
text = "  hello world  "  
uppercase_text = text.upper()  
print(uppercase_text)  
# Output: '  HELLO WORLD  '
```

Multiple Words

```
text = "good morning everyone"  
uppercase_text = text.upper()  
print(uppercase_text)  
# Output: 'GOOD MORNING EVERYONE'
```

Already Uppercase

```
text = "ALREADY UPPERCASE"  
uppercase_text = text.upper()  
print(uppercase_text)  
# Output: 'ALREADY UPPERCASE'
```

lower()

- Converts all letters in a string to lowercase.

Simple Lowercase Conversion

```
# Convert a string to lowercase.  
text = "HELLO WORLD"  
lowercase_text = text.lower()  
print(lowercase_text)  
# Output: 'hello world'
```

Mixed Case to Lowercase

```
# Convert a mixed case string to lowercase.  
text = "PyThON Is FuN"  
lowercase_text = text.lower()  
print(lowercase_text)  
# Output: 'python is fun'
```

Numbers and Special Characters

```
# Numbers and special characters remain unchanged.  
text = "Welcome to 2024!"  
lowercase_text = text.lower()  
print(lowercase_text)  
# Output: 'welcome to 2024!'
```

Empty String Handling

```
# Applying lower() on an empty string.  
text = ""  
lowercase_text = text.lower()  
print(lowercase_text)  
# Output: ''
```

Concept Practice Task

String with Leading and Trailing Spaces

```
text = "    HELLO WORLD    "  
lowercase_text = text.lower()  
  
print(lowercase_text)  
# Output: '    hello world    '
```

Multiple Words Handling

```
text = "GOOD MORNING EVERYONE"  
lowercase_text = text.lower()  
  
print(lowercase_text)  
# Output: 'good morning everyone'
```

Already Lowercase

```
text = "already lowercase"  
lowercase_text = text.lower()  
  
print(lowercase_text)  
# Output: 'already lowercase' (no change)
```


strip()

- Remove **Leading** and **Trailing** Spaces
- Removes any extra spaces from the start and end of the string.

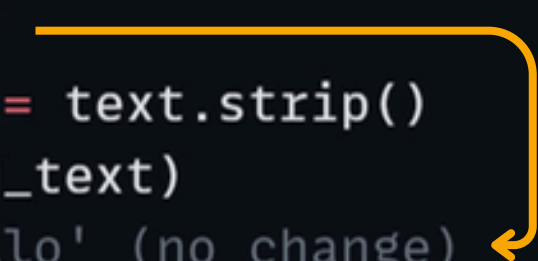
Basic Strip

```
# Remove leading and trailing spaces.  
text = "__hello world__"  
stripped_text = text.strip()  
print(stripped_text)  
# Output: 'hello world'
```



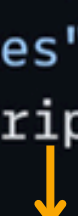
No Leading or Trailing Spaces

```
# String without leading or trailing spaces.  
text = "hello"  
stripped_text = text.strip()  
print(stripped_text)  
# Output: 'hello' (no change)
```



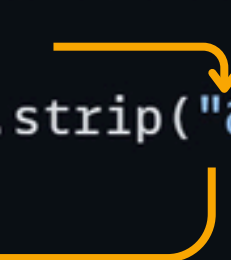
Only Leading Spaces

```
# Remove leading spaces from the string.  
text = "   leading spaces"  
stripped_text = text.strip()  
print(stripped_text)  
# Output: 'leading spaces'
```



Custom Character Strip

```
# Remove specified characters from the string.  
text = "@@@hello@@@"  
stripped_text = text.strip("@")  
print(stripped_text)  
# Output: 'hello'
```



Concept Practice Task

Only Trailing Spaces

```
text = "trailing spaces "  
stripped_text = text.strip()  
  
print(stripped_text)  
# Output: 'trailing spaces'
```

Empty String

```
text = ""  
stripped_text = text.strip()  
  
print(stripped_text)  
# Output: ''
```

Strip with Special Characters

```
text = "!!hello!!"  
stripped_text = text.strip()  
  
print(stripped_text)  
# Output: '!!hello!!'
```

Strip with Newlines and Tabs

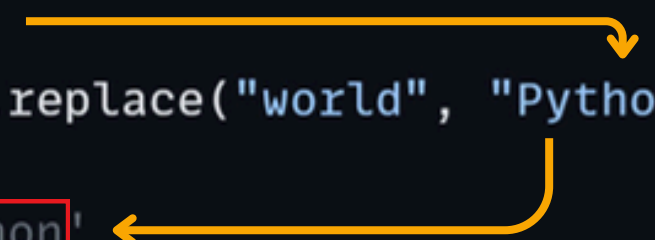
```
text = "\n\t hello world \t\n"  
stripped_text = text.strip()  
  
print(stripped_text)  
# Output: 'hello world'
```

replace(old, new)

- Replaces a substring with another substring:

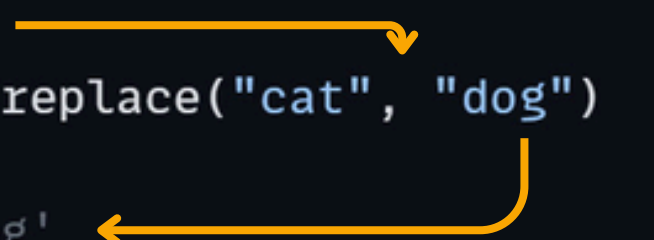
Basic Replacement

```
# Replace a substring with another substring.
text = "hello world"
replaced_text = text.replace("world", "Python")
print(replaced_text)
# Output: 'hello Python'
```

A yellow arrow points from the word 'world' in the string "hello world" to the word 'Python' in the output string 'hello Python'. The word 'Python' in the output is highlighted with a red box.

Replace Multiple Occurrences

```
# Replace multiple occurrences of a substring.
text = "cat cat cat"
replaced_text = text.replace("cat", "dog")
print(replaced_text)
# Output: 'dog dog dog'
```

A yellow arrow points from the first occurrence of 'cat' in the string "cat cat cat" to the first occurrence of 'dog' in the output string 'dog dog dog'. The word 'dog' in the output is highlighted with a red box.

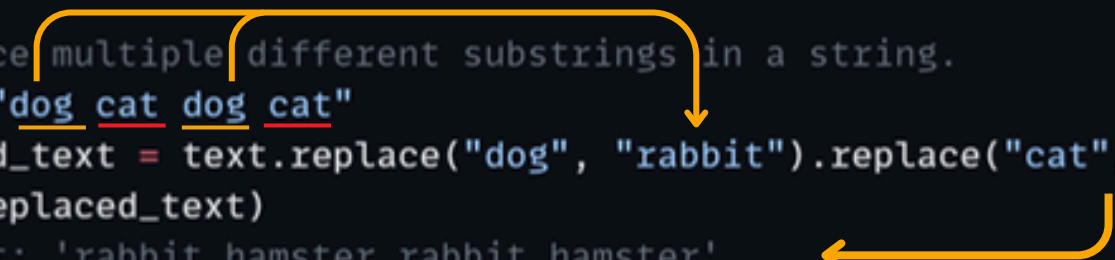
Case-Sensitive Replacement

```
# Replacement is case-sensitive.
text = "Hello World"
replaced_text = text.replace("world", "Python")
print(replaced_text)
# Output: 'Hello World' ( no change )
```

A yellow arrow points from the word 'world' in the string "Hello World" to the word 'World' in the output string 'Hello World'. The word 'World' in the output is highlighted with a red box.

Replace Multiple Substrings

```
# Replace multiple different substrings in a string.
text = "dog cat dog cat"
replaced_text = text.replace("dog", "rabbit").replace("cat", "hamster")
print(replaced_text)
# Output: 'rabbit hamster rabbit hamster'
```

Two yellow arrows point from the words 'dog' and 'cat' in the string "dog cat dog cat" to the words 'rabbit' and 'hamster' in the output string 'rabbit hamster rabbit hamster'. The words 'rabbit' and 'hamster' in the output are highlighted with red boxes.

Concept Practice Task

GamutX

Replace with Empty String

```
text = "remove spaces"
replaced_text = text.replace("spaces", "")

print(replaced_text)
# Output: 'remove '
```

Replace Non-Existing Substring

```
text = "hello"
replaced_text = text.replace("world", "Python")

print(replaced_text)
# Output: 'hello' (no change)
```

Replace with Special Characters

```
text = "hello @world@"
replaced_text = text.replace("@", "#")

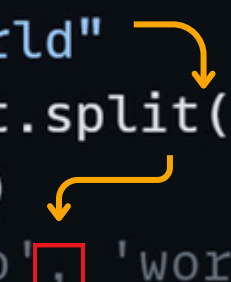
print(replaced_text)
# Output: 'hello #world#'
```


split(separator)

- Splits a string into a list of substrings based on a specified delimiter (separator).

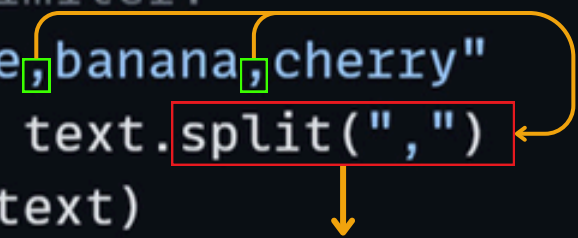
Basic Split

```
# Default space delimiter.  
text = "hello world"  
split_text = text.split()  
print(split_text)  
# Output: ['hello', 'world']
```



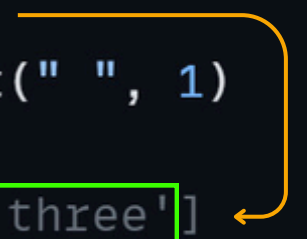
Split with Custom Delimiter

```
# Custom delimiter.  
text = "apple,banana,cherry"  
split_text = text.split(",")  
print(split_text)  
# Output: ['apple', 'banana', 'cherry']
```



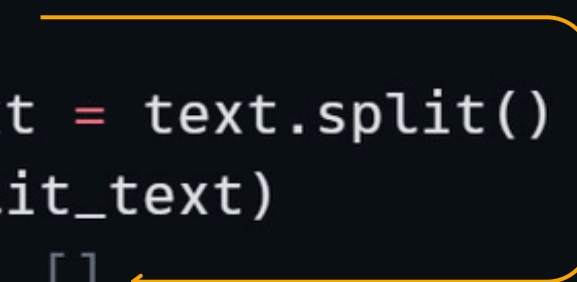
Split with Limited Splits

```
# Limit the number of splits to 1.  
text = "one two three"  
split_text = text.split(" ", 1)  
print(split_text)  
# Output: ['one', 'two three']
```



Split an empty string.

```
text = ""  
split_text = text.split()  
print(split_text)  
# Output: []
```



Concept Practice Task

No Separator provided

```
sentence = "Python is fun"
words = sentence.split()
print(words)
# Output: ['Python', 'is', 'fun']
```

Custom Separator

```
data = "apple,banana,orange"
fruits = data.split(',')
print(fruits)
# Output: ['apple', 'banana', 'orange']
```

Handling Multiple Consecutive Delimiters

```
text = "a,,b,,c"
split_text = text.split(",")

print(split_text)
# Output: ['a', '', 'b', '', 'c']
```

join()

- The .join() method is used to **combine a list** (or sequence) of strings into one single string using a separator.

Basic Join

```
# Join substrings using a specified delimiter.  
text = "hello"  
joined_text = "-".join(text)  
print(joined_text)  
# Output: 'h-e-l-l-o'
```

Joining with an Empty String

```
text = "join"  
joined_text = "".join(text)  
print(joined_text)  
# Output: 'join'
```

Joining a String with Custom Delimiters

```
# Each character is separated by a custom delimiter.  
text = "test"  
delimiter = "|"   
joined_text = delimiter.join(text)  
print(joined_text)  
# Output: 't|e|s|t'
```

Joining with a Newline Character

```
# Join substrings with a newline delimiter.  
substrings = "line1", "line2", "line3"  
joined_text = "\n".join(substrings)  
print(joined_text)  
# Output:  
# line1  
# line2  
# line3
```

find()

- Returns the index of the first occurrence of the specified substring.
- If the substring is not found, it returns -1.

Basic Find

```
# Find first occurrence of a substring.  
text = "hello world"  
index = text.find("world")  
print(index)  
# Output: 6
```

It return the Index value

Substring Not Found

```
# Return -1 if the substring is not found.  
text = "hello"  
index = text.find("world")  
print(index)  
# Output: -1
```

Find with Start Position

```
# Find starting from a specified position.  
text = "hello Python"  
index = text.find("hello", 6)  
print(index)  
# Output: -1
```

What to Find

From where to Start Finding

Find with End Position

```
# Find substring within a specified range.  
text = "Python, hello"  
index = text.find("hello", 0, 5)  
print(index)  
# Output: -1
```

What to Find

Start Position

End Position

Concept Practice Task

Multiple Occurrences

```
text = "apple banana apple"
index = text.find("apple")

print(index)
# Output: 0
```

Find on Empty String

```
text = ""
index = text.find("test")

print(index)
# Output: -1
```

Case Sensitivity

```
text = "Hello World"
index = text.find("world")

print(index)
# Output: -1
```

count(substring)

- Counts how many times a substring appears in the string.

Basic Count

```
# Count the no of occurrences of a substring.  
text = "hello world"  
count = text.count("o")  
print(count)  
# Output: 2
```

Count Substring Not Found

```
# Return 0 if the substring is not found.  
text = "hello world"  
count = text.count("z")  
print(count)  
# Output: 0
```

Count with Start and End Position

```
# Count occurrences specified range.  
text = "hello hello hello"  
count = text.count("hello", 6, 12)  
print(count)  
# Output: 1
```

What to count

Start Position

End Position

Count with Empty String

```
# Count occurrences of an empty string  
# (counts as one more than the number of characters).  
text = "hello"  
count = text.count("")  
print(count)  
# Output: 6
```

1. Before `h`
2. Between `h` and `e`
3. Between `e` and `l`
4. Between `l` and `l`
5. Between `l` and `o`
6. After `o`

Concept Practice Task

Count with Case Sensitivity

```
text = "Hello hello"  
count = text.count("hello")
```

```
print(count)
```

```
# Output: 1
```

startswith()

- Checks if a string starts with a given prefix.
- Returns True if it does, otherwise False.

Basic Check

```
text = "hello world"
starts = text.startswith("hello")
print(starts)
# Output: True
```

Prefix Not Present

```
text = "hello world"
starts = text.startswith("world")

print(starts)
# Output: False
```

With All Arguments

```
text = "Learning Python is fun"

print(text.startswith("Python", 9, 15))
# Output: True
```

Concept Practice Task

Empty String Check

```
text = ""
starts = text.startswith("")

print(starts)
# Output: True
```

Using with start Index

```
text = "Hello, world!"
print(text.startswith("world", 7))
#Output: True
```

Case Sensitivity

```
text = "Hello World"
starts = text.startswith("hello")

print(starts)
# Output: False
```


endswith()

- The .endswith() method checks if a string ends with a specific suffix.
- It returns True if the string ends with the given value, otherwise False.

Basic Check

```
# String ends with the specified suffix.  
text = "hello world"  
ends = text.endswith("world")  
print(ends)  
# Output: True
```



It returns True if
String found at end

Suffix Not Present

```
# string does not end with the specified suffix.  
text = "hello world"  
ends = text.endswith("hello")  
print(ends)  
# Output: False
```



It returns False if String
not found at end

Case Sensitivity

```
# String ends with a suffix in a case-sensitive manner.  
text = "Hello World"  
ends = text.endswith("world")  
print(ends)  
# Output: False
```



World != world

Empty String Check

```
# empty string ends with an empty suffix (always True).  
text = ""  
ends = text.endswith("")  
print(ends)  
# Output: True
```



Concept Practice Task

Using with start and end Arguments

```
text = "Python programming"
print(text.endswith("Python", 0, 6))
#Output: True
```

with All Arguments

```
text = "Learning Python is fun"
print(text.endswith("Python", 0, 15))
#Output: True
```

using len() function within

```
text = "Python programming is powerful"
print(text.endswith("powerful", 0, len(text)))
#Output: True
```


Summary of Common String Methods

- **len():** Returns the length of a string.
- **upper():** Converts the string to uppercase.
- **lower():** Converts the string to lowercase.
- **strip():** Removes leading and trailing whitespace.
- **replace(old, new):** Replaces a substring within the string.
- **split(separator):** Splits the string into a list based on a delimiter.
- **join():** Joins a list of strings into one string.
- **find(substring):** Returns the index of the first occurrence of a substring.
- **count(substring):** Counts how many times a substring occurs.
- **startswith():** checks if the string begins with the specified prefix.
- **endswith() :**checks if the string ends with the specified suffix.

Thank You

